08. BERT PRE TRAINING

14 March 2026 13:20

(Devlin et al 2019)

Bidirectional Encoder Representations from transformers

BERT → designed to pretrain deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers

↳ so a pretrained BERT can be fine tuned with just one additional output layer to create SOTA models for a wide range of tasks, such as question answering etc without substantial modifications

bidirectional

↳ in other bidirectional models (BLSTM)

Two separate passes

left → right } then concatenate
right → left }

↳ BERT → no two passes → every token sends every other token simultaneously

$h_i = \text{Attention}(x_1, x_2, \ldots x_n)$

Information flows globally in one layer

Before BERT → task specific Architectures

After → large pretraining + minimal task adaptation

Autoregressive models → must learn language structure

fully bidirectional prediction is hard

↳ left context + right context + token itself

prediction becomes trivial, The model learns Identity Mapping & not Language Structure

The core constraint BERT must solve is → Allow bidirectional context
BUT prevent seeing the answer } training objective design problem

in Autoregressive model

$P(x_t | x_1, x_{t-1})$

token + cannot see the future

A causality constraint

↳ create asymmetric information flow

past → future future X → past

restricts representations

BERT claims → remove asymmetry

transformers architecture already allows full bidirectional attention

Challenge → training without leakage

pretraining objective is the innovation

We want

context → predict missing info } this forces model to encode semantic relationships b/w tokens

not

token → see itself

Framework

pretrain large representation model

↓

Add small task head

↓

Fine-tune entire network } transfer learning paradigm

The model doesn't know things like grammar etc
it only looks at tokens and relations

Yet grammar emerges from training objective

Introduction

Language model pretraining → effective for many NLP tasks (natural language inference, paraphrasing etc)

(Named entity Recognition, question answering) — token level

sentence level tasks

Two strategies to apply pretrained language representations to down stream tasks

↳ fine tuning strategy
↳ feature based strategy } two approaches during pretraining use unidirectional LM's to learn general language representations

McCormick

pretrained model → produces embeddings

↓

use embeddings as features in another model

pipeline

sentence → pretrained encoder → embeddings

↓

task specific architecture

ELMo → feature based → uses task specific architectures that include pretrained representations as additional features

McCormick

Pretrained Model

↓

attach small task head

↓

train entire network on task } We adapt the whole model instead of embeddings

GPT → fine tuning

↳ introduces minimal task specific parameters

↳ trained on down stream tasks by simply fine tuning all pretrained parameters

At that time unidirectionality of standard language models is a limitation

↳ BERT addresses it through "Masked Language Model" (MLM) pretraining objective

↓

randomly masks some of the tokens from the input

↳ Objective is to predict the original vocabulary id of the masked word based only on its context

in addition to MLM we also use a next sentence prediction task that jointly pretrains text-pair representations → learns sentence level representations

→ they demonstrate importance of bidirectional pre-training for language representations

→ Show pretrained represent^ns reduce the need for many heavily engineered task specific architectures

The point is "Less Task Specific Engineering"

related work

→ Unsupervised feature based approaches

they discuss → early word embeddings → core idea → word fixed vector → vectors are context independent

→ Left to Right LMs → directionality constraint

→ Sentence level objectives → predict next, surrounding sentences, rank them

discourse level representations

→ ELMO

→ Left to Right LMs → directionality combined

→ Sentence level objectives → predict next, surrounding sentences, rank them

→ ELMO

↳ forward LSTM → L → R
backward LSTM → R → L ⇒ $h_i = [\overrightarrow{h_i}, \overleftarrow{h_i}]$

↳ contextual dependent embeddings

even though ELMO bidirectional

forward uses only past
Backward uses only future

and then they are concatenated

Interaction happens after encoding, not inside each layer

→ unsupervised fine tuning approaches

↳ Advantage was few parameters need to be learned from scratch

task specific output layer is learnt from scratch → not params are fine tuned not frozen

→ Transfer learning from supervised data

pretrained network is updated during training

BERT

two steps → pre training → model is trained on unlabeled data over different pre training tasks → learns general language representations

→ fine tuning → model is first initialized with pre trained params & all of params are fine tuned using labeled data from downstream tasks

↳ Adapt the same parameters to a specific task

each task has separate fine tuned models
they are initialized with same pre trained parameters

Unified architecture across different tasks

Minimal difference between Pretrained architecture and final downstream architecture

All parameters are fine tuned → not just the output layer

Model Architecture → Based on transformers

No of layers (transformer blocks) → L
Hidden size → H
No of self attention heads → A

$BERT_{base}$ = L=12 H=768 A=12 110M params
$BERT_{large}$ = L=24 H=1024 A=16 340M params

BERT uses a transformer encoder stack

Input/Output Representations

sentence → Arbitrary span of contiguous text

sequence → input token sentence, which may be a single sentence or two sentences packed together

↳ first token of every sequence is always a special classification token [CLS]

↳ purpose Aggregate representation of entire sequence

final hidden vector used for classification tasks

Sentence pairs are packed together into a single sequence

↳ we write sentences in two ways

↳ first we separate them with [SEP] token

↳ we add a learned embedding to every token indicating whether it belongs to sentence A/B

for token i: $E_i = E_i^{token} + E_i^{segment} + E_i^{position}$ → encodes token position

used mainly for not sentence level prediction tasks

input vector is sum of these three embeddings

Input Embedding 'E'

final hidden vector special [CLS] token as $C \in \mathbb{R}^H$

final hidden vector of ith input token as $T_i \in \mathbb{R}^H$

for a given token
its input repres^n is constructed
by summing corresponding token, segment and position embeddings

Input	[CLS] my dog [SEP] my cat [SEP]
Token Embeddings	$E_{[CLS]}$ E_{my} — — —
Segment Embeddings	E_A E_A — — —
Position embeddings	E_0 E_1 — — — —

3.1 Pre training BERT

Task #1 Masked LM

training process

1 Randomly choose 15% of tokens

2 For each chosen token

Case	Action
80%	replace with [MASK]
10%	replace with random token
10%	Keep unchanged

if they only use [MASK] it would appear in real tasks

the mismatch b/w pre training & fine tuning

Hence the introduction of these

Case	Action
80%	replace with [MASK]
10%	replace with random token
10%	keep unchanged

if they only use [MASK] it would appear in real texts → mismatch b/w pre training & fine tuning

Hence they introduce noise

For each masked position:

Transformer produces hidden vector h_i

Then prediction $p(w|h_i) = \text{softmax}(Wh_i)$ — maps hidden vector to vocabulary logits

Training objective: cross-entropy loss over vocabulary

They feed hidden vector of masked tokens into softmax → but loss is computed only on masked tokens, not all tokens

Task #2 → Next Sentence Prediction

Training example
Sentence A
Sentence B

Two cases

Label	Meaning
IsNext	B is actual next sentence
NotNext	B is random sentence

Probability 50% IsNext
50% NotNext

Prediction Mechanism (use CLS representation)

Classifier $P(\text{label}) = \text{softmax}(WC)$

So CLS becomes a sentence pair representation

They use document level corpora so that sentence order is preserved for NSP

NSP eg [CLS] sentence A tokens [SEP] sentence B tokens [SEP]

Segment embeddings make the sentences segment A, segment B

After passing through transformer layers, every token has a hidden vector

Token	Hidden vector
[CLS]	C
token 1	T_1
token 2	T_2
⋮	⋮

$C \in \mathbb{R}^H$ → C vector is the representation of the entire sequence

Encodes information from both sentences because attention allows tokens to interact with whole sequence

Model applies a binary classifier on the [CLS] vector

$P(\text{isNext}|C) = \text{softmax}(WC)$ — C: final hidden vector of CLS

Classifier outputs isNext or notNext

during attention encoder signals like topic continuity, entity overlap, discourse flow

Classification weights learn only to collect sequence level information

Fine tuning → simply attach a task-specific head

eg Sentence classification
[CLS] → classifier

Token level tasks (NER)
token representation → classifier

Question Answering
Start position
End position
predicted using token vectors

pre training → expensive
fine tuning → cheap

BERT uses sentence pairs
encoder using self attention

BERT simply concatenates sentences and lets self-attention handle interaction

Earlier systems used two encoders + cross attention

fine tuning (Mathematically)

θ_{BERT} → these params include
attention weights
feed forward layers
embeddings
layer norms

during fine tuning we add a small task specific head θ_{task}

total parameters
$\theta = (\theta_{BERT}, \theta_{task})$

training objective becomes $\min_{\theta_{BERT}, \theta_{task}} L_{task}$

both sets of params are updated using gradient descent

What actually changes during fine tuning

Before fine tuning
BERT wts = pretrained
task head = random initialization

During fine tuning
task head learns quickly
BERT weights adjust slowly

Model shifts from general language knowledge to task specific decision boundaries

Modern methods use two extra tricks
layer wise learning rate decay
warmup schedule
} to stabilize fine tuning

They wanted to prove through experiments
i Bidirectional pretraining improves representations
ii One architecture can solve many NLP tasks
iii Pretraining + fine tuning beats task specific systems

from GLUE Benchmark they observed (Scaling hypothesis support)

larger model → Richer representation → better downstream results

Used SQuAD Question Answering for BERT's ability on span extraction tasks

No task specific architecture needed — Same architecture + different output head

SWAG (Common sense reasoning task)

BERT's bidirectionality outperforms ELMO, GPT — different NLP tasks

Key pattern across all experiments

All tasks use same core architecture

Only the output layer changes

Task type	output layer
Classification	[CLS] → softmax
QA span extraction	token vectors → start/end
Sentence similarity	[CLS] → regression

Before BERT
each NLP task → custom architecture

After BERT
general language encoder → small task head

MLM forces embeddings to encode contextual meaning

MLM forces tokens to encode contextual meaning
[CLS] learns to aggregate sentence level meaning

Ablation studies
↳ isolate → three design choices
i pre training objectives → Bidirectional MLM is critical, NSP also helps
ii Model size → supports scaling laws

Why Bidirectionality matters?
LTR + RTL → computationally expensive

Ablation studies

↳ isolate → three design choices

i pre training objectives → Bidirectional MLM is critical, NSP also helps

ii Model size → supports scaling laws

iii fine tuning vs feature based approach

↳ BERT performs well on both

BERT essentially learns a language representation function

$f_\theta(x_1 \ldots x_n)$

which produces

contextual token embeddings

+ a sequence embedding

Additional Ablation studies done →

effect of no of training steps

Different Masking procedures

↳ purpose of masking strategies was to reduce the mismatch between pre-training and fine-tuning

Why Bidirectionality matters?

$\frac{LTR + RTL}{\text{concatenate}}$ → computationally expensive

interactions btwn left & right contexts happens only at top

in BERT left, Right interaction occurs at every transformer layer